

PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Taha Qadri

BCADS27

Roll no. - 1250258472

3rd semester

Babu Banarasi Das University

Lucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as \$gt, \$lt, \$eq, \$gte, \$ne, \$and, \$or and \$not. Every operation is documented with the exact command and a screenshot of the output produced in the MongoDB shell (mongosh). The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 49 students across the DS, AI, BCA and CS departments (13 DS, 12 AI, 12 BCA and 12 CS at the time of insertion), and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and a screenshot of the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document (roll number 202) into the students collection.

Code:

```
db.students.insertOne({
  rollNo: 202,
  name: "Rohan",
  age: 19,
  department: "DS",
  marks: 76
})
```

Output:

```
> db.students.insertOne({ rollNo: 202, name: "Rohan", age: 19, department: "DS", marks: 76 })
{
  acknowledged: true,
  insertedId: ObjectId('6ac5f1d2f252e6b438269e0d')
}
```

Figure 1: Output of insertOne()

2. Insert Many

Inserts multiple student documents (roll numbers 203 to 250) into the collection at the same time.

Code:

```
db.students.insertMany([
  { rollNo: 203, name: "Meera", age: 21, department: "AI", marks: 91 },
  { rollNo: 204, name: "Kunal", age: 20, department: "BCA", marks: 68 },
  { rollNo: 205, name: "Aisha", age: 22, department: "CS", marks: 87 },
  { rollNo: 206, name: "Dev", age: 19, department: "DS", marks: 79 },
  { rollNo: 207, name: "Naina", age: 20, department: "AI", marks: 94 },
  { rollNo: 208, name: "Arnav", age: 21, department: "BCA", marks: 72 },
  { rollNo: 209, name: "Sara", age: 20, department: "CS", marks: 89 },
  { rollNo: 210, name: "Kabir", age: 19, department: "DS", marks: 65 },
  { rollNo: 211, name: "Ishita", age: 22, department: "AI", marks: 96 },
  { rollNo: 212, name: "Vivek", age: 20, department: "BCA", marks: 81 },
  { rollNo: 213, name: "Zoya", age: 21, department: "CS", marks: 74 },
  { rollNo: 214, name: "Harsh", age: 19, department: "DS", marks: 88 },
  { rollNo: 215, name: "Ritika", age: 20, department: "AI", marks: 83 },
  { rollNo: 216, name: "Adil", age: 22, department: "BCA", marks: 70 },
  { rollNo: 217, name: "Kavya", age: 21, department: "CS", marks: 92 },
  { rollNo: 218, name: "Naman", age: 20, department: "DS", marks: 77 },
  { rollNo: 219, name: "Alisha", age: 19, department: "AI", marks: 85 },
  { rollNo: 220, name: "Sahil", age: 21, department: "BCA", marks: 69 },
  { rollNo: 221, name: "Mansi", age: 20, department: "CS", marks: 90 },
  { rollNo: 222, name: "Yash", age: 22, department: "DS", marks: 73 },
  { rollNo: 223, name: "Hina", age: 19, department: "AI", marks: 97 },
  { rollNo: 224, name: "Aarav", age: 20, department: "BCA", marks: 80 },
  { rollNo: 225, name: "Sana", age: 21, department: "CS", marks: 86 },
  { rollNo: 226, name: "Mohit", age: 20, department: "DS", marks: 71 },
  { rollNo: 227, name: "Diya", age: 19, department: "AI", marks: 93 },
  { rollNo: 228, name: "Rajat", age: 22, department: "BCA", marks: 67 },
  { rollNo: 229, name: "Anaya", age: 20, department: "CS", marks: 84 },
  { rollNo: 230, name: "Faizan", age: 21, department: "DS", marks: 78 },
  { rollNo: 231, name: "Pihu", age: 19, department: "AI", marks: 90 },
  { rollNo: 232, name: "Abhay", age: 20, department: "BCA", marks: 75 },
  { rollNo: 233, name: "Ira", age: 21, department: "CS", marks: 95 },
  { rollNo: 234, name: "Manav", age: 22, department: "DS", marks: 66 },
  { rollNo: 235, name: "Zara", age: 19, department: "AI", marks: 82 },
  { rollNo: 236, name: "Tushar", age: 20, department: "BCA", marks: 79 },
  { rollNo: 237, name: "Anvi", age: 21, department: "CS", marks: 91 },
  { rollNo: 238, name: "Sameer", age: 20, department: "DS", marks: 70 },
  { rollNo: 239, name: "Myra", age: 19, department: "AI", marks: 87 },
  { rollNo: 240, name: "Aman", age: 22, department: "BCA", marks: 64 },
  { rollNo: 241, name: "Neha", age: 20, department: "CS", marks: 88 },
  { rollNo: 242, name: "Rishi", age: 21, department: "DS", marks: 76 },
  { rollNo: 243, name: "Ishaan", age: 19, department: "AI", marks: 98 },
  { rollNo: 244, name: "Palak", age: 20, department: "BCA", marks: 83 },
  { rollNo: 245, name: "Rehan", age: 22, department: "CS", marks: 72 },
  { rollNo: 246, name: "Tanya", age: 21, department: "DS", marks: 89 },
  { rollNo: 247, name: "Arman", age: 20, department: "AI", marks: 81 },
  { rollNo: 248, name: "Simran", age: 19, department: "BCA", marks: 74 },
  { rollNo: 249, name: "Kartik", age: 21, department: "CS", marks: 93 },
  { rollNo: 250, name: "Aditi", age: 20, department: "DS", marks: 86 }
])
```

Output:

```
> db.students.insertMany([ { rollNo: 203, ... }, ... { rollNo: 250, ... } ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac5f1d2f252e6b438269e0e'),
    '1': ObjectId('6ac5f1d2f252e6b438269e0f'),
    '2': ObjectId('6ac5f1d2f252e6b438269e10'),
    '3': ObjectId('6ac5f1d2f252e6b438269e11'),
    '4': ObjectId('6ac5f1d2f252e6b438269e12'),
    // ... 43 more entries ('5' to '47') not shown
  }
}
```

Figure 2: Output of insertMany() - the shell returns one ObjectId per inserted document (list shortened here)

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 202 are updated from 76 to 80.

Code:

```
db.students.updateOne(
  { rollNo: 202 },
  { $set: { marks: 80 } }
)
```

Output:

```
> db.students.updateOne({ rollNo: 202 }, { $set: { marks: 80 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 3: Output of updateOne()

4. Update Many

Updates all documents that match the given condition - here, every student in the DS department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
  { department: "DS" },
  { $set: { status: "Active" } }
)
```

Output:

```
> db.students.updateMany({ department: "DS" }, { $set: { status: "Active" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 13,
  modifiedCount: 13,
  upsertedCount: 0
}
```

Figure 4: Output of updateMany() - all 13 DS documents modified

Verification of the updates:

```
db.students.findOne({ rollNo: 202 })
```

Output:

```
> db.students.findOne({ rollNo: 202 })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0d'),
  rollNo: 202,
  name: 'Rohan',
  age: 19,
  department: 'DS',
  marks: 80,
  status: 'Active'
}
```

Figure 5: Roll no. 202 now shows marks: 80 and the new status field

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 250 is removed.

Code:

```
db.students.deleteOne({ rollNo: 250 })
```

Output:

```
> db.students.deleteOne({ rollNo: 250 })
{
  acknowledged: true,
  deletedCount: 1
}
```

Figure 6: Output of deleteOne()

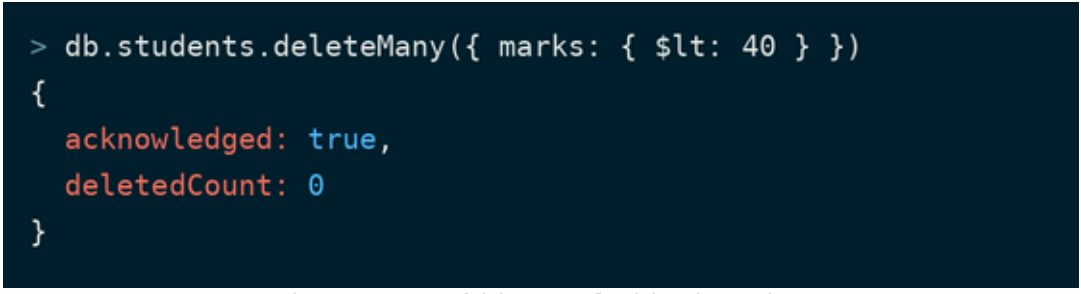
6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed. Since the lowest marks in the collection are 64, no document matches and nothing is deleted.

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:



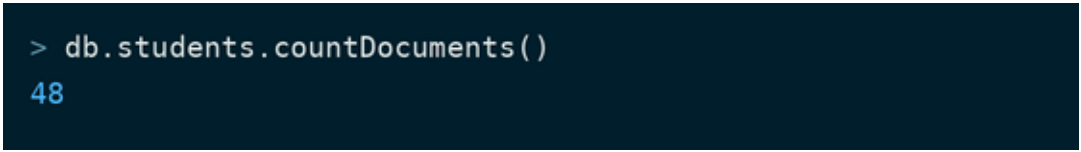
```
> db.students.deleteMany({ marks: { $lt: 40 } })
{
  acknowledged: true,
  deletedCount: 0
}
```

Figure 7: Output of deleteMany() - deletedCount is 0

Total documents after all update and delete operations:

```
db.students.countDocuments()
```

Output:



```
> db.students.countDocuments()
48
```

Figure 8: 48 documents remain (49 inserted - 1 deleted)

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records. For each query, the screenshot shows the shell output (the first three matching documents are captured; the shell prints the rest in the same format) and the table lists every matching record.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
> db.students.find({ marks: { $gt: 80 } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0e'),
  rollNo: 203,
  name: 'Meera',
  age: 21,
  department: 'AI',
  marks: 91
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e10'),
  rollNo: 205,
  name: 'Aisha',
  age: 22,
  department: 'CS',
  marks: 87
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e12'),
  rollNo: 207,
  name: 'Naina',
  age: 20,
  department: 'AI',
  marks: 94
}
```

Figure 9: Marks greater than 80 - 26 documents matched (first 3 shown)

All matching records (26):

Roll No	Name	Age	Dept	Marks
203	Meera	21	AI	91
205	Aisha	22	CS	87
207	Naina	20	AI	94
209	Sara	20	CS	89
211	Ishita	22	AI	96
212	Vivek	20	BCA	81
214	Harsh	19	DS	88
215	Ritika	20	AI	83
217	Kavya	21	CS	92
219	Alisha	19	AI	85
221	Mansi	20	CS	90
223	Hina	19	AI	97
225	Sana	21	CS	86

Roll No	Name	Age	Dept	Marks
227	Diya	19	AI	93
229	Anaya	20	CS	84
231	Pihu	19	AI	90
233	Ira	21	CS	95
235	Zara	19	AI	82
237	Anvi	21	CS	91
239	Myra	19	AI	87
241	Neha	20	CS	88
243	Ishaan	19	AI	98
244	Palak	20	BCA	83
246	Tanya	21	DS	89
247	Arman	20	AI	81
249	Kartik	21	CS	93

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
> db.students.find({ marks: { $lt: 80 } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0f'),
  rollNo: 204,
  name: 'Kunal',
  age: 20,
  department: 'BCA',
  marks: 68
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e11'),
  rollNo: 206,
  name: 'Dev',
  age: 19,
  department: 'DS',
  marks: 79,
  status: 'Active'
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e13'),
  rollNo: 208,
  name: 'Arnav',
  age: 21,
  department: 'BCA',
  marks: 72
}
```

Figure 10: Marks less than 80 - 20 documents matched (first 3 shown)

All matching records (20):

Roll No	Name	Age	Dept	Marks
204	Kunal	20	BCA	68
206	Dev	19	DS	79
208	Arnav	21	BCA	72
210	Kabir	19	DS	65
213	Zoya	21	CS	74
216	Adil	22	BCA	70
218	Naman	20	DS	77
220	Sahil	21	BCA	69
222	Yash	22	DS	73
226	Mohit	20	DS	71

Roll No	Name	Age	Dept	Marks
228	Rajat	22	BCA	67
230	Faizan	21	DS	78
232	Abhay	20	BCA	75
234	Manav	22	DS	66
236	Tushar	20	BCA	79
238	Sameer	20	DS	70
240	Aman	22	BCA	64
242	Rishi	21	DS	76
245	Rehan	22	CS	72
248	Simran	19	BCA	74

Marks equal to 85:

Code:

```
db.students.find({ marks: { $eq: 85 } })
```

Output:

```
> db.students.find({ marks: { $eq: 85 } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e1e'),
  rollNo: 219,
  name: 'Alisha',
  age: 19,
  department: 'AI',
  marks: 85
}
```

Figure 11: Marks equal to 85 - 1 documents matched (first 3 shown)

All matching records (1):

Roll No	Name	Age	Dept	Marks
219	Alisha	19	AI	85

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
> db.students.find({ marks: { $gte: 80 } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0d'),
  rollNo: 202,
  name: 'Rohan',
  age: 19,
  department: 'DS',
  marks: 80,
  status: 'Active'
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0e'),
  rollNo: 203,
  name: 'Meera',
  age: 21,
  department: 'AI',
  marks: 91
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e10'),
  rollNo: 205,
  name: 'Aisha',
  age: 22,
  department: 'CS',
  marks: 87
}
```

Figure 12: Marks greater than or equal to 80 - 28 documents matched (first 3 shown)

All matching records (28):

Roll No	Name	Age	Dept	Marks
202	Rohan	19	DS	80
203	Meera	21	AI	91
205	Aisha	22	CS	87
207	Naina	20	AI	94
209	Sara	20	CS	89
211	Ishita	22	AI	96
212	Vivek	20	BCA	81
214	Harsh	19	DS	88
215	Ritika	20	AI	83
217	Kavya	21	CS	92
219	Alisha	19	AI	85
221	Mansi	20	CS	90
223	Hina	19	AI	97
224	Aarav	20	BCA	80

Roll No	Name	Age	Dept	Marks
225	Sana	21	CS	86
227	Diya	19	AI	93
229	Anaya	20	CS	84
231	Pihu	19	AI	90
233	Ira	21	CS	95
235	Zara	19	AI	82
237	Anvi	21	CS	91
239	Myra	19	AI	87
241	Neha	20	CS	88
243	Ishaan	19	AI	98
244	Palak	20	BCA	83
246	Tanya	21	DS	89
247	Arman	20	AI	81
249	Kartik	21	CS	93

Marks not equal to 85:**Code:**

```
db.students.find({ marks: { $ne: 85 } })
```

Output:

```
> db.students.find({ marks: { $ne: 85 } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0d'),
  rollNo: 202,
  name: 'Rohan',
  age: 19,
  department: 'DS',
  marks: 80,
  status: 'Active'
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0e'),
  rollNo: 203,
  name: 'Meera',
  age: 21,
  department: 'AI',
  marks: 91
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0f'),
  rollNo: 204,
  name: 'Kunal',
  age: 20,
  department: 'BCA',
  marks: 68
}
```

Figure 13: Marks not equal to 85 - 47 documents matched (first 3 shown)

All matching records (47):

Roll No	Name	Age	Dept	Marks
202	Rohan	19	DS	80
203	Meera	21	AI	91
204	Kunal	20	BCA	68
205	Aisha	22	CS	87
206	Dev	19	DS	79
207	Naina	20	AI	94
208	Arnav	21	BCA	72
209	Sara	20	CS	89
210	Kabir	19	DS	65
211	Ishita	22	AI	96
212	Vivek	20	BCA	81
213	Zoya	21	CS	74
214	Harsh	19	DS	88
215	Ritika	20	AI	83
216	Adil	22	BCA	70
217	Kavya	21	CS	92
218	Naman	20	DS	77
220	Sahil	21	BCA	69
221	Mansi	20	CS	90
222	Yash	22	DS	73
223	Hina	19	AI	97
224	Aarav	20	BCA	80
225	Sana	21	CS	86
226	Mohit	20	DS	71

Roll No	Name	Age	Dept	Marks
227	Diya	19	AI	93
228	Rajat	22	BCA	67
229	Anaya	20	CS	84
230	Faizan	21	DS	78
231	Pihu	19	AI	90
232	Abhay	20	BCA	75
233	Ira	21	CS	95
234	Manav	22	DS	66
235	Zara	19	AI	82
236	Tushar	20	BCA	79
237	Anvi	21	CS	91
238	Sameer	20	DS	70
239	Myra	19	AI	87
240	Aman	22	BCA	64
241	Neha	20	CS	88
242	Rishi	21	DS	76
243	Ishaan	19	AI	98
244	Palak	20	BCA	83
245	Rehan	22	CS	72
246	Tanya	21	DS	89
247	Arman	20	AI	81
248	Simran	19	BCA	74
249	Kartik	21	CS	93

Marks greater than 70 AND age less than 23:

Code:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 23 } }
  ]
})
```

Output:

```
> db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 23 } }
  ]
})
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0d'),
  rollNo: 202,
  name: 'Rohan',
  age: 19,
  department: 'DS',
  marks: 80,
  status: 'Active'
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0e'),
  rollNo: 203,
  name: 'Meera',
  age: 21,
  department: 'AI',
  marks: 91
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e10'),
  rollNo: 205,
  name: 'Aisha',
  age: 22,
  department: 'CS',
  marks: 87
}
```

Figure 14: Marks greater than 70 AND age less than 23 - 40 documents matched (first 3 shown)

All matching records (40):

Roll No	Name	Age	Dept	Marks
202	Rohan	19	DS	80
203	Meera	21	AI	91
205	Aisha	22	CS	87
206	Dev	19	DS	79
207	Naina	20	AI	94
208	Arnav	21	BCA	72
209	Sara	20	CS	89
211	Ishita	22	AI	96
212	Vivek	20	BCA	81
213	Zoya	21	CS	74
214	Harsh	19	DS	88
215	Ritika	20	AI	83
217	Kavya	21	CS	92
218	Naman	20	DS	77
219	Alisha	19	AI	85
221	Mansi	20	CS	90
222	Yash	22	DS	73
223	Hina	19	AI	97
224	Aarav	20	BCA	80
225	Sana	21	CS	86

Roll No	Name	Age	Dept	Marks
226	Mohit	20	DS	71
227	Diya	19	AI	93
229	Anaya	20	CS	84
230	Faizan	21	DS	78
231	Pihu	19	AI	90
232	Abhay	20	BCA	75
233	Ira	21	CS	95
235	Zara	19	AI	82
236	Tushar	20	BCA	79
237	Anvi	21	CS	91
239	Myra	19	AI	87
241	Neha	20	CS	88
242	Rishi	21	DS	76
243	Ishaan	19	AI	98
244	Palak	20	BCA	83
245	Rehan	22	CS	72
246	Tanya	21	DS	89
247	Arman	20	AI	81
248	Simran	19	BCA	74
249	Kartik	21	CS	93

Department is CS OR AI:

Code:

```
db.students.find({
  $or: [
    { department: "CS" },
    { department: "AI" }
  ]
})
```

Output:

```
> db.students.find({
  $or: [
    { department: "CS" },
    { department: "AI" }
  ]
})
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0e'),
  rollNo: 203,
  name: 'Meera',
  age: 21,
  department: 'AI',
  marks: 91
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e10'),
  rollNo: 205,
  name: 'Aisha',
  age: 22,
  department: 'CS',
  marks: 87
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e12'),
  rollNo: 207,
  name: 'Naina',
  age: 20,
  department: 'AI',
  marks: 94
}
```

Figure 15: Department is CS OR AI - 24 documents matched (first 3 shown)

All matching records (24):

Roll No	Name	Age	Dept	Marks
203	Meera	21	AI	91
205	Aisha	22	CS	87
207	Naina	20	AI	94
209	Sara	20	CS	89
211	Ishita	22	AI	96
213	Zoya	21	CS	74
215	Ritika	20	AI	83
217	Kavya	21	CS	92
219	Alisha	19	AI	85
221	Mansi	20	CS	90
223	Hina	19	AI	97
225	Sana	21	CS	86

Roll No	Name	Age	Dept	Marks
227	Diya	19	AI	93
229	Anaya	20	CS	84
231	Pihu	19	AI	90
233	Ira	21	CS	95
235	Zara	19	AI	82
237	Anvi	21	CS	91
239	Myra	19	AI	87
241	Neha	20	CS	88
243	Ishaan	19	AI	98
245	Rehan	22	CS	72
247	Arman	20	AI	81
249	Kartik	21	CS	93

Marks NOT greater than 80:**Code:**

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Output:

```
> db.students.find({ marks: { $not: { $gt: 80 } } })
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0d'),
  rollNo: 202,
  name: 'Rohan',
  age: 19,
  department: 'DS',
  marks: 80,
  status: 'Active'
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e0f'),
  rollNo: 204,
  name: 'Kunal',
  age: 20,
  department: 'BCA',
  marks: 68
}
{
  _id: ObjectId('6ac5f1d2f252e6b438269e11'),
  rollNo: 206,
  name: 'Dev',
  age: 19,
  department: 'DS',
  marks: 79,
  status: 'Active'
}
```

Figure 16: Marks NOT greater than 80 - 22 documents matched (first 3 shown)

All matching records (22):

Roll No	Name	Age	Dept	Marks
202	Rohan	19	DS	80
204	Kunal	20	BCA	68
206	Dev	19	DS	79
208	Arnav	21	BCA	72
210	Kabir	19	DS	65
213	Zoya	21	CS	74
216	Adil	22	BCA	70
218	Naman	20	DS	77
220	Sahil	21	BCA	69
222	Yash	22	DS	73
224	Aarav	20	BCA	80

Roll No	Name	Age	Dept	Marks
226	Mohit	20	DS	71
228	Rajat	22	BCA	67
230	Faizan	21	DS	78
232	Abhay	20	BCA	75
234	Manav	22	DS	66
236	Tushar	20	BCA	79
238	Sameer	20	DS	70
240	Aman	22	BCA	64
242	Rishi	21	DS	76
245	Rehan	22	CS	72
248	Simran	19	BCA	74

8. MongoDB Operator Summary

Operator	Meaning	Example
\$gt	Greater than	marks: { \$gt: 80 }
\$lt	Less than	marks: { \$lt: 80 }
\$eq	Equal to	marks: { \$eq: 85 }
\$gte	Greater than or equal to	marks: { \$gte: 80 }
\$ne	Not equal to	marks: { \$ne: 85 }
\$and	All conditions must be true	marks > 70 AND age < 23
\$or	At least one condition must be true	CS OR AI
\$not	Negates a condition	NOT (marks > 80)

Total dataset: 49 students inserted (Roll No. 202-250); 48 remain after deleting Roll No. 250.

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 49 student records.
- updateOne() and updateMany() with the \$set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators (\$gt, \$lt, \$eq, \$gte, \$ne) and logical operators (\$and, \$or, \$not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 49 student records. The updateOne operation successfully changed one student's marks (76 to 80), while updateMany correctly modified all 13 DS-department documents to add an "Active" status. The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0, leaving 48 documents in the collection. The query operations correctly filtered students by marks and department using comparison and logical operators: 26 students scored above 80 while 22 did not, 20 students scored below 80 while 28 scored 80 or more, exactly one student scored 85, and 24 students belong to the CS or AI departments. These counts are consistent with each other ($26 + 22 = 48$), demonstrating that MongoDB's query engine returns accurate, condition-based results.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: <https://www.mongodb.com/docs/>
- MongoDB CRUD Operations: <https://www.mongodb.com/docs/manual/crud/>
- MongoDB Query Operators: <https://www.mongodb.com/docs/manual/reference/operator/query/>

- mongosh (MongoDB Shell) Documentation:
<https://www.mongodb.com/docs/mongodb-shell/>

THANK YOU!